502094 - Software Deployment, Operations And Maintenance

Module 10: Continuous Delivery & Release Management

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Introduction to Continuous Delivery (CD)

- Position of **Continuous Delivery (CD)** in the DevOps lifecycle
- How CD connects **CI pipelines** with **real deployments**
- Why CD reduces risk and speeds up delivery
- Overview of:
  - CD concepts
  - CD pipelines
  - Release management strategies

What Is Continuous Delivery?

- **Continuous Delivery (CD)**:
  - Practice of ensuring software is always in a **deployable state**
  - Every successful build can be deployed to production at any time
- Key characteristics:
  - Automated build, test, and packaging
  - Low-friction, repeatable deployments
  - Manual decision: *when* to deploy, not *if* we can deploy

Continuous Deployment vs Continuous Delivery

- **Continuous Delivery**:
  - CI builds and tests
  - CD pipeline prepares releases
  - **Deployment to production is a manual decision**
- **Continuous Deployment**:
  - Everything in CD +
  - **Automatic deployment** to production after passing all checks
- In practice:
  - Many teams do Continuous Delivery for production
  - Continuous Deployment is often used for internal or low-risk services

CI vs CD vs Continuous Deployment

- **CI (Continuous Integration)**:
  - Focus: integrating code changes, running tests
  - Output: **validated artifacts**
- **CD (Continuous Delivery)**:
  - Focus: preparing deployable releases
  - Output: **release candidates** ready for production
- **Continuous Deployment**:
  - Focus: automatically pushing changes to production
  - Output: **running software in production** updated frequently

Modern DevOps Pipeline

- Typical pipeline stages:
  - **Commit** → **Build** → **Test** → **Artifact** → **Release** → **Deploy**
- CI usually covers:
  - Build + Test
- CD covers:
  - Artifact storage
  - Release creation
  - Deployment automation
  - Rollback strategies

Why CD Reduces Risk

- Small, frequent deployments:
  - Smaller change sets
  - Easier debugging
- Repeatable, tested deployment process:
  - Fewer manual errors
- Automated checks in each stage:
  - Block unsafe releases early
- Clear audit trail:
  - Which build, which commit, who approved

Prerequisites for CD

- Strong **CI foundation**:
  - Reliable test suite
  - Fast and reproducible builds
- Good **observability**:
  - Logging, metrics, and health checks
- Clear **versioning and artifact strategy**
- Culture:
  - Treat deployments as **boring and routine**, not as special events

Core Principles of Continuous Delivery

- Build once, promote everywhere
- Reproducible, immutable artifacts
- Versioning and tagging of every release
- Environment parity (dev/staging/prod)
- Safe, automated, repeatable deployments
- Rollback-friendly processes

Build Once, Promote Everywhere

- Anti-pattern:
  - Rebuilding source code separately for dev, staging, prod
- Principle:
  - Build **once** in CI
  - Store artifact in a registry/repository
  - Deploy the **same artifact** to dev → staging → prod
- Benefits:
  - Reproducibility
  - Reduced “works in staging but not prod” issues

Reproducible, Immutable Artifacts

- Artifacts examples:
  - **Spring Boot**: `booking-service-1.3.0.jar`
  - **Node.js**: `dist/` directory, zipped build
  - **Docker**: `mvmanh/booking-api:1.3.0`
- **Immutable**:
  - Once created, not modified
  - New releases → new versions/tags
- Enables:
  - Reliable rollback
  - Auditable deployments

Versioning & Tagging Strategies

- **Semantic Versioning**:
  - `MAJOR.MINOR.PATCH` → `1.2.3`
  - Major: breaking changes
  - Minor: new features, backward-compatible
  - Patch: bug fixes
- **Git tags**:
  - `v1.2.3` associated with a specific commit
- **Docker tags**:
  - `1.2.3`, `1.2`, `latest`, and commit-based tags like `sha-8f3a1e2`

Environment Parity

- **Goal**:
  - Dev, staging, and production should be **as similar as possible**
- Differences:
  - Configuration: URLs, credentials, secrets
  - Scale: Number of instances, CPU/RAM
- Same:
  - Application version
  - Deployment method (JAR/Docker)
  - Observability and health checks

Safe, Automated, Repeatable Deployments

- Ideal deployment:
  - Click a button or run a workflow → same result every time
- Avoid:
  - Manual copying via SCP
  - Manual edits on production servers
- Use:
  - Scripts (Bash, PowerShell)
  - Infrastructure-as-Code (Terraform, Ansible)
  - GitHub Actions workflows

Rollback-Friendly Deployment Processes

- Every deployment must be:
  - **Reversible**
  - **Predictable**
- Rollback strategies:
  - Deploy previous JAR version
  - Run previous Docker image tag
  - Switch traffic back to previous environment (blue-green)
- CD pipeline should:
  - Make rollbacks as simple as deploys

CD Pipeline Architecture

- Stages in a typical CD pipeline:
  - Build (via CI)
  - Packaging
  - Artifact storage
  - Release creation
  - Deployment automation
- Implemented as:
  - One or more GitHub Actions workflows
  - Plus configuration in registries and servers

Build & Packaging Stages

- Build:
  - Handled by CI: `./mvnw package`, `npm run build`
- Packaging:
  - Zip Node.js `dist/` and `node_modules` (if needed)
  - Build Spring Boot fat JAR
  - Build Docker image using Dockerfile
- Output:
  - A **single artifact** per service and version

Artifact Storage Options

- **GitHub Releases**:
  - Store JARs, zipped builds, changelogs
- **Container registries**:
  - Docker Hub: `mvmanh/booking-api:1.3.0`
  - AWS ECR: `123456789012.dkr.ecr.ap-southeast-1.amazonaws.com/booking-api:1.3.0`
- **Object storage**:
  - AWS S3 buckets:
    - `s3://artifacts-tdtu-devops/booking-api/1.3.0/booking-api-1.3.0.jar`

Deployment Automation

- Automation should:
  - Fetch artifact (JAR or Docker image)
  - Stop old version
  - Start new version
  - Validate with health checks
- Tools:
  - GitHub Actions + SSH
  - Ansible playbooks
  - Kubernetes manifests + kubectl (next modules)
- We will focus on: GitHub Actions + EC2 deployments

Security Considerations in CD

- Use **GitHub Secrets** for:
  - SSH private keys
  - Registry credentials
  - Cloud access keys
- Principle of **least privilege**:
  - CI/CD credentials can: Deploy; Read artifacts
  - But not: Perform unrelated admin tasks
- Always:
  - Avoid storing secrets in Git
  - Limit who can trigger production deployments

Release Artifacts & Versioning

- Release management:
  - Assign versions
  - Create Git tags
  - Upload artifacts
  - Generate release notes
- Purpose:
  - Clear traceability from:
    - Commit → Artifact → Deployment

Semantic Versioning in Practice

- Example:
  - `booking-api`:
    - `1.4.0` — Add payment integration
    - `1.4.1` — Fix bug in payment validation
    - `2.0.0` — Major API change, breaking clients
- Map Git tags: `v1.4.0`, `v1.4.1` `v2.0.0`
- Docker images:
  - `mvmanh/booking-api:1.4.1`
  - `mvmanh/booking-api:1.4`
  - `mvmanh/booking-api:latest`

Using Git Tags as Release Identifiers

- Create tag locally:

```
git tag -a v1.4.1 -m "Release v1.4.1 - bug fixes"
git push origin v1.4.1
```

- GitHub Actions:
  - Can be triggered on:

```yaml
on:
  push:
    tags:
      - "v*.*.*"
```

- Advantage:
  - CI/CD pipeline tightly bound to specific version

Docker Tagging Strategies

- Combine:
  - Human-friendly tags:
    - `1.4.1`, `1.4`, `latest`
  - Machine-friendly tags:
    - `sha-8f3a1e2`
- In pipeline:

```
docker tag mvmanh/booking-api:${GITHUB_SHA} mvmanh/booking-api:1.4.1
docker tag mvmanh/booking-api:${GITHUB_SHA} mvmanh/booking-api:latest
```

- Allows:
  - Rollback to `1.4.0`
  - Debugging using commit-based tag

Generating Changelogs Automatically

- Goal:
  - Summarize changes between `v1.4.0` and `v1.4.1`
- Approaches:
  - Conventional commits + changelog generator
  - Git log-based scripts:

    ```
    git log v1.4.0..v1.4.1 --oneline
    ```

- Use changelog content:
  - In GitHub Release notes
  - In internal release documentation

Uploading Artifacts to GitHub Releases

```yaml
- name: Create GitHub Release
  uses: softprops/action-gh-release@v2
  with:
    tag_name: ${{ github.ref_name }}
    name: "Release ${{ github.ref_name }}"
    files: |
      target/booking-api-*.jar
      dist/frontend-bundle.zip
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

- Advantage:
  - Single page with:
    - Version, Description
    - Downloadable artifacts

Continuous Delivery with GitHub Actions

- We will focus on:
  - CD workflows structure
  - Automated release creation
  - Docker image build & publish
  - Deployments to EC2 with SSH
  - Multi-environment deployments (dev/staging/prod)

Structure of CD Workflows

- Common triggers:
  - `workflow_dispatch` (manual)
  - `push` to tags (e.g., `v1.4.1`)
  - `release` events
- CD workflows usually:
  - Depend on CI workflows
  - Reuse artifacts (JAR, Docker images)
- Environments:
  - `dev`, `staging`, `production` configured in GitHub

Example CD Workflow Trigger

```yaml
name: CD - Deploy Booking API
on:
  workflow_dispatch:
    inputs:
      environment:
        description: "Target environment"
        required: true
        default: "staging"
        type: choice
        options:
          - dev
          - staging
          - production
```

- Allows: Manual start of CD pipeline; Choosing environment at runtime

Using GitHub Environments

- Define environments in GitHub:
  - `dev`, `staging`, `production`
- For each environment:
  - Separate secrets:
    - `EC2_HOST_DEV`, `EC2_HOST_STAGING`, `EC2_HOST_PROD`
  - Protection rules:
    - Require approval before deploying to production
- In workflow:

```
environment: ${{ inputs.environment }}
```

Automating Release Creation

- Goal:
  - When tag `v1.4.1` is pushed:
    - Run tests (CI)
    - Create GitHub Release
    - Attach artifacts
- Trigger:

```yaml
on:
  push:
    tags:
      - "v*.*.*"
```

Auto-Generating GitHub Releases

```yaml
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build JAR
        run: ./mvnw -B clean package

      - name: Create Release
        uses: softprops/action-gh-release@v2
        with:
          tag_name: ${{ github.ref_name }}
          name: "Booking API ${{ github.ref_name }}"
          files: target/booking-api-*.jar
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Auto-Generating GitHub Releases

- Result:
  - Release page with:
    - JAR
    - Version
    - Release notes

Build & Publish Docker Images (Overview)

- Steps in CD context:
  - Use CI build to:
    - Build Docker image
    - Run tests
  - Use CD workflow to:
    - Tag image appropriately
    - Push to Docker Hub / ECR
    - Trigger deployment jobs

Multi-Stage Dockerfile Review

```
FROM node:20-alpine AS build
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /usr/src/app
COPY --from=build /usr/src/app/dist ./dist
ENV NODE_ENV=production
CMD ["node", "dist/server.js"]
```

- Advantages:
  - Smaller runtime image, Faster startup, Reduced attack surface

Build & Push Docker Image in Workflow

```yaml
- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKER_USERNAME }}
    password: ${{ secrets.DOCKER_PASSWORD }}

- name: Build image
  run: docker build -t mvmanh/booking-api:${{ github.sha }} .

- name: Tag versions
  run: |
    docker tag mvmanh/booking-api:${{ github.sha }} mvmanh/booking-api:${{ github.ref_name }}
    docker tag mvmanh/booking-api:${{ github.sha }} mvmanh/booking-api:latest

- name: Push images
  run: |
    docker push mvmanh/booking-api:${{ github.sha }}
    docker push mvmanh/booking-api:${{ github.ref_name }}
    docker push mvmanh/booking-api:latest
```

Deploying Node.js / Spring Boot to EC2 via SSH

- **Integrated Practical Requirement #1**
- Idea:
  - Build artifact in CI
  - CD workflow:
    - Connect to EC2 via SSH
    - Upload artifact
    - Restart application process
    - Validate with health check
- Secrets:
  - `EC2_HOST`, `EC2_USER`, `EC2_SSH_PRIVATE_KEY`, `APP_DIR`

Preparing SSH Key on Runner

```yaml
- name: Configure SSH key
  run: |
    mkdir -p ~/.ssh
    echo "${{ secrets.EC2_SSH_PRIVATE_KEY }}" > ~/.ssh/id_rsa
    chmod 600 ~/.ssh/id_rsa
    ssh-keyscan -H $EC2_HOST >> ~/.ssh/known_hosts
  env:
    EC2_HOST: ${{ secrets.EC2_HOST }}
```

- Purpose:
  - Set up private key securely
  - Avoid host key prompts using `ssh-keyscan`

Example: Deploy Spring Boot App to EC2

```yaml
- name: Deploy Spring Boot App to EC2
  env:
    EC2_HOST: ${{ secrets.EC2_HOST }}
    EC2_USER: ${{ secrets.EC2_USER }}
    APP_DIR: ${{ secrets.APP_DIR }}
  run: |
    scp -i ~/.ssh/id_rsa target/booking-api-*.jar $EC2_USER@$EC2_HOST:$APP_DIR/
    ssh -i ~/.ssh/id_rsa $EC2_USER@$EC2_HOST << 'EOF'
      cd /home/ubuntu/app
      pkill -f "java -jar" || true
      nohup java -jar booking-api-*.jar --spring.profiles.active=prod > app.log 2>&1 &
      echo "App deployed successfully!"
    EOF
```

Teaching Points: Multi-Line Scripts & Heredoc

- `run: |`:
  - Allows multi-line shell script in a step
- Heredoc:

```
ssh user@host << 'EOF'
  cd /home/ubuntu/app
  pkill -f "java -jar" || true
  nohup java -jar booking-api-*.jar > app.log 2>&1 &
EOF
```

- Advantages:
  - Write multiple remote commands as a single, readable block
  - Easier to maintain than many `ssh` calls

Restarting Java or Node.js Processes Cleanly

- Java (Spring Boot):
  - `pkill -f "java -jar"` to stop old instance
  - `nohup java -jar app.jar &` to start in background
- Node.js:
  - Use `pm2` or `systemd`:

```
pm2 restart booking-api || pm2 start dist/server.js --name booking-api
```

- Always:
  - Redirect stdout/stderr to log file
  - Avoid orphan processes

Deploy Docker-Based Applications to EC2 From Registry

- **Integrated Practical Requirement #2**
- Pipeline:
  - CI builds & pushes Docker image
  - CD job:
    - SSH to EC2
    - `docker pull` new image
    - Stop & remove old container
    - Run new container with proper ports and env vars

Example: Deploy Docker Image to EC2

```yaml
- name: Deploy Docker Image to EC2
  env:
    EC2_HOST: ${{ secrets.EC2_HOST }}
    EC2_USER: ${{ secrets.EC2_USER }}
    IMAGE_TAG: ${{ github.sha }}
  run: |
    ssh -o StrictHostKeyChecking=no $EC2_USER@$EC2_HOST << 'EOF'
      docker pull mvmanh/booking-api:${IMAGE_TAG}
      docker stop booking-api || true
      docker rm booking-api || true
      docker run -d --name booking-api \
        -p 80:3000 \
        -e NODE_ENV=production \
        mvmanh/booking-api:${IMAGE_TAG}
      echo "Container deployed successfully!"
    EOF
```

Teaching Points: Commit-Based Tags & Rollback

- Using `${{ github.sha }}` as `IMAGE_TAG`:
  - Guarantees **immutable** mapping to a specific commit
- Simplified rollback:
  - Re-run deployment with previous `IMAGE_TAG`
  - Or maintain `previous_tag` and `current_tag` variables
- Containers:
  - Make it easy to run multiple versions side-by-side

Multi-Environment Deployments

- Typical pattern:
  - **Dev**:
    - Automatic deployment on merge
  - **Staging**:
    - Manual approval required
  - **Production**:
    - Manual approval + branch protection
- Use environment-specific:
  - Hosts
  - Secrets
  - Configurations

Per-Environment Secrets in GitHub Actions

- Define secrets for each environment:
  - `EC2_HOST_DEV`, `EC2_HOST_STAGING`, `EC2_HOST_PROD`
  - `DOCKER_REPO_DEV`, `DOCKER_REPO_PROD`
- In workflow:

```
environment: ${{ inputs.environment }}
```

- GitHub resolves:
  - Appropriate secrets based on chosen environment

Deployment Strategies (High-Level Overview)

- Strategies to manage risk:
  - **Rolling deployments**
  - **Blue-Green deployments**
  - **Canary releases**
- Each strategy:
  - Affects how traffic is shifted
  - Influences ease of rollback

Rolling Deployments

- Process:
  - Gradually replace old instances with new ones
- VM / EC2 example:
  - Update instances in an Auto Scaling Group
- Kubernetes example:
  - Deployment rolling update with `maxUnavailable` and `maxSurge`
- Advantages:
  - No downtime if configured correctly
- Challenge:
  - Handling incompatible schema changes

Blue-Green Deployments

- Two environments:
  - **Blue** (current production)
  - **Green** (new version)
- Steps:
  i. Deploy new version to Green
  ii. Run tests and health checks
  iii. Switch traffic from Blue → Green (e.g., via Nginx or load balancer)
- Rollback:
  - Switch back to Blue quickly

Canary Releases (Overview)

- Gradual rollout:
  - Start with small percentage of users
  - Monitor errors and performance
  - Increase traffic gradually if healthy
- Typically implemented with:
  - Feature flags
  - Traffic-splitting load balancers
  - Service meshes (e.g., Istio)
- Useful for:
  - High-risk changes

Automated Rollbacks

- Goal:
  - Automatically revert when deployment is unhealthy
- Signals:
  - Failing health checks
  - Increased error rates
  - Timeouts or high latency
- Actions:
  - Redeploy previous Docker image tag
  - Switch Nginx upstream back to previous group
  - Rollback Kubernetes Deployment to previous ReplicaSet

Example: Docker Image Rollback

```
# On EC2 host
docker stop booking-api || true
docker rm booking-api || true
docker run -d --name booking-api   -p 80:3000   mvmanh/booking-api:1.4.0
```

- CD workflow can:
  - Expose a manual `rollback` input
  - Execute a similar script referencing previous version

Environment Configuration & Secrets Management

- Key concerns:
  - Storing configuration safely
  - Separating configuration for dev/staging/prod
  - Avoiding secret leaks in logs and repos
- Tools:
  - GitHub Environment Secrets
  - `.env` files for local dev (never commit)
  - External secret stores (e.g., AWS SSM Parameter Store)

Never Store Secrets in Git

- Secrets **must not** be:
  - Committed to Git
  - Embedded in Docker images
  - Logged in plain text
- Use:
  - `secrets.*` in GitHub Actions
  - Cloud-native secret managers
- Hardening:
  - Rotate secrets regularly
  - Limit access to those who need them

GitHub Actions Secrets vs Environment Secrets

- **Repository-level secrets**:
  - Shared across all workflows
- **Environment secrets**:
  - Scoped to GitHub Environments (dev, staging, prod)
- Prefer:
  - Environment secrets for deployment credentials
  - Enables different hosts and tokens per environment

External Secret Providers

- Example: AWS SSM Parameter Store
  - Store `/booking-api/prod/DB_PASSWORD`
- In deployment script:
  - Fetch at runtime:

```
aws ssm get-parameter \
  --name "/booking-api/prod/DB_PASSWORD" \
  --with-decryption \
  --query "Parameter.Value" \
  --output text
```

- Benefit:
  - Centralized management
  - Fine-grained IAM access control

Observability for CD Pipelines

- After each deployment, we must:
  - Validate that the system is healthy
  - Monitor for regressions
- Observability tools:
  - Logs
  - Metrics
  - Traces
- CD pipeline should:
  - Include basic checks (e.g., `curl` health endpoint)

Health Checks After Deployment

- Node.js API:
  - `/health` endpoint:

```
app.get("/health", (req, res) => {
  res.status(200).json({ status: "ok" });
});
```

- Spring Boot:
  - Use **Actuator**:
    - `/actuator/health`
- In CD workflow:

```
- name: Verify health endpoint
  run: curl -f http://$EC2_HOST/health
```

Application Logs vs System Logs

- **Application logs**:
  - Business events and errors
  - Example:
    - `app.log` from Spring Boot or Node.js
- **System logs**:
  - OS and services:
    - `journalctl` for systemd services
    - Docker logs for containers
- During CD:
  - Use logs to debug failed deployments
  - Integrate with log platforms (CloudWatch, ELK, Loki)

Optional Integrations: Observability Stack

- Metrics:
  - Prometheus:
    - Scrapes metrics from apps
  - Grafana:
    - Dashboards for deployments and performance
- Cloud-native:
  - AWS CloudWatch: Logs, metrics, alarms
- CD pipeline:
  - Can notify:
    - Slack/Teams when deployment is done
    - When alarms trigger

Hands-On Labs: Full CD Experience

- Lab 1 — Build & Release Artifact
- Lab 2 — Build & Push Docker Image
- Lab 3 — Deploy Node.js / Spring Boot to EC2 via SSH
- Lab 4 — Deploy Docker Image from Registry to EC2
- Lab 5 — Staging Deployment with Approval Gate

Lab 1 — Build & Release Artifact

- Objective:
  - Build Node.js or Spring Boot app
  - Upload artifacts to GitHub Release
- Tasks:
  - Trigger workflow on tag push
  - Build and test application
  - Generate JAR or zipped dist
  - Create GitHub Release with attached artifacts
- Skills:
  - Tagging
  - Release automation

Lab 2 — Build & Push Docker Image

- Objective:
  - Use GitHub Actions to build and push Docker image
- Tasks:
  - Build multi-stage image
  - Tag with semantic version + commit SHA
  - Push to Docker Hub
- Skills:
  - Dockerfile optimization
  - Registry authentication with secrets
  - Tag management

Lab 3 — Deploy Node.js / Spring Boot to EC2 via SSH

- Objective:
  - Implement full CD pipeline via SSH
- Tasks:
  - Configure SSH key with GitHub Secrets
  - Upload JAR or Node.js build to EC2
  - Restart application using nohup or systemd
  - Validate deployment with `curl` health check
- Skills:
  - Shell scripting
  - Remote process management
  - Log inspection

Lab 4 — Deploy Docker Image from Registry to EC2

- Objective: Deploy Dockerized app from registry to EC2
- Tasks:
  - Pull new image by SHA tag
  - Stop/remove existing container
  - Start new container and expose port
  - Confirm smooth startup and health
- Skills:
  - Container lifecycle management
  - Rollback to previous tags
  - Security: using registry credentials via secrets

Lab 5 — Staging Deployment with Approval Gate

- Objective:
  - Introduce approval to staging deployments
- Tasks:
  - Configure GitHub Environment `staging` with required reviewers
  - Create CD workflow using `environment: staging`
  - Trigger staging deployment, approve, then deploy
  - Simulate broken deployment, roll back, and re-deploy fixed version
- Skills:
  - GitHub Environments
  - Approval workflows
  - Rollback procedures

Best Practices for Continuous Delivery

- Always deploy from **artifacts**, not from source
- Use clear, consistent versioning
- Design every deployment with **rollback** in mind
- Deploy to staging before production
- Use environment-specific secrets
- Keep deployment scripts DRY, modular, and stored in Git
- Never rebuild Docker images during deployment

Summary & Next Steps

- CD and release management:
  - Turn reliable CI into reliable deployments
- You learned:
  - CD principles and pipeline architecture
  - Artifact and version management
  - Practical GitHub Actions for EC2 and Docker deployments
- Next:
  - Combine these CD techniques with:
    - Kubernetes
    - Infrastructure-as-Code
    - Advanced observability